Application of Numerical Linear Algebra to PageRank in Search Engine Systems

MTH 4311: Numerical Analysis

Final Project Report

Artem Okhten
Applied Mathematics

Professor Jian Du

May 6, 2026

Abstract

In this project I study PageRank as application of numerical linear algebra to search engine systems. PageRank is usually introduced as web ranking algorithm, but mathematically it becomes fixed-point problem, eigenvector problem, and sparse linear algebra problem.

The web is represented as directed graph with pages as nodes and hyperlinks as directed edges. From this graph, I construct sparse adjacency matrix A, transition matrix S, and PageRank vector r. Main numerical method is power iteration. The implementation uses sparse matrix structure, builds transition matrix by diagonal scaling $S = AD^{-1}$, and handles dangling node without filling dense columns.

For experiment, I use small graph with $n = 10$ pages and 23 directed links. Page 10 is dangling node. With $\alpha = 0.85$ and tolerance 10^{-10}, sparse power iteration converged in 27 iterations. Final L_1 change was 9.474×10^{-11}, and final PageRank vector summed to 1.000000000000. Direct linear solve was also used as verification for small graph, and difference between direct solve and power iteration was 5.539×10^{-11}.

The main numerical finding is that PageRank is not only ranking idea. It is also example of how eigenvalue methods, sparse matrices, convergence, stopping tolerance, and floating-point error appear inside computer system.

1 Introduction

Search engine does not only store web pages. It also needs to decide which pages are more important when many pages match same query. One classical way to measure this importance is PageRank, introduced by Page, Brin, Motwani, and Winograd to rank web pages using link structure of the web [1].

Basic idea of PageRank is simple. A page becomes important when other important pages link to it. This creates recursive definition, because rank of one page depends on ranks of other pages. In mathematical form, this becomes eigenvector problem or sparse linear system. Because of this, PageRank is good example of numerical linear algebra inside real computer system.

This topic fits Numerical Analysis because it uses matrix formulation, iterative methods, eigenvalue problems, convergence, stopping tolerance, sparse matrices, and numerical stability. In real search engine, number of pages can be extremely large. Direct matrix methods are not practical there. Instead, iterative method such as power iteration is used to approximate ranking vector.

The purpose of this report is to explain how PageRank is formulated mathematically, how it is solved numerically, and what numerical challenges appear when method is used on graph data. I use small web graph example because it is easier to inspect. However, numerical structure is same one used in much larger systems.

2 Problem Description and Mathematical Formulation

2.1 Web Graph Model

The web can be modeled as directed graph. Each page is represented by node, and each hyperlink from one page to another is represented by directed edge. If page j links to page i, then user currently on page j may move to page i by following that link.

Suppose there are n pages. Link structure can be represented by adjacency matrix A, where

$$A_{ij} = \begin{cases} 1, & \text{if page } j \text{ links to page } i, \\ 0, & \text{otherwise.} \end{cases}$$

This column convention is useful because each column describes outgoing links from one page. If page j has d_j outgoing links, then probability of moving from page j to page i is

$$S_{ij} = \frac{A_{ij}}{d_j}.$$

Matrix S is hyperlink transition matrix. For non-dangling pages, each column of S should sum to one, so matrix is column-stochastic.

Computationally, transition matrix should not be built by modifying one sparse column at a time. If graph is large, repeated column updates can cause unnecessary memory reallocation. Better way is to normalize non-dangling columns using sparse diagonal scaling.

Let D be diagonal matrix of outgoing degrees:

$$D = \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{bmatrix}.$$

For non-dangling pages, transition matrix is constructed as

$$S = AD^{-1}.$$

In MATLAB implementation, D^{-1} is built using `spdiags`. Adjacency matrix A is also assembled from index vectors instead of repeated element-wise sparse assignment. MATLAB stores sparse matrices internally in Compressed Sparse Column format. Modifying sparse entries one by one can force memory reallocation, while assembling matrix from index vectors and using diagonal scaling matches sparse storage better.

2.2 Dangling Nodes

A dangling node is page with no outgoing links. If page j has no outgoing links, then $d_j = 0$, so column $A(:, j)$ cannot be divided by d_j. If this issue is ignored, transition matrix will not be column-stochastic, and PageRank vector will not behave like probability distribution.

Simple approach would be to replace each dangling column by uniform vector $1/n$. This is acceptable for very small classroom example, but it is not efficient for large sparse graphs. Filling even one dangling column creates n nonzero entries, which destroys sparsity.

Because of this, MATLAB implementation handles dangling nodes implicitly. Let $\mathcal{D}$ be set of dangling pages. At iteration k, total rank mass located on dangling pages is

$$m_D^{(k)} = \sum_{j \in \mathcal{D}} r_j^{(k)}.$$

Instead of explicitly filling dangling columns, this mass is redistributed uniformly during update:

$$r^{(k+1)} = \alpha S r^{(k)} + \alpha m_D^{(k)} v + (1 - \alpha) v,$$

where v is teleportation vector. This keeps S sparse and still preserves probability interpretation of PageRank model.

2.3 Damping Factor and Google Matrix

If users only followed links, process could get trapped inside closed group of pages. To avoid this, PageRank introduces damping factor α, usually around 0.85. With probability α, user follows link. With probability $1 - \alpha$, user teleports to random page.

In this project, I use uniform teleportation vector

$$v = \frac{1}{n} \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}.$$

Here, $r^{(k)}$ is rank vector at iteration k. Each component of r represents long-run probability of being on that page. The iteration from previous section combines three parts: link-following, dangling-node mass redistribution, and teleportation.

If full transition matrix including dangling-node correction is written as S_{full}, then PageRank can also be written as

$$r = \alpha S_{\text{full}} r + (1 - \alpha) v.$$

Rearranging gives linear system

$$(I - \alpha S_{\text{full}}) r = (1 - \alpha) v.$$

This form is important because it shows that PageRank is not only web ranking idea. It is also numerical linear algebra problem.

3 Numerical Discretization and Computational Algorithm

3.1 Why This Is Numerical Problem

Unlike differential equation problem, PageRank does not need spatial or time discretization. Discretization here comes from converting real web browsing system into finite directed graph. Continuous behavior of user moving through web is modeled as discrete-time Markov chain with finite number of states. Each state represents one web page.

After web pages are represented as nodes and hyperlinks are represented as matrix entries, ranking problem becomes finite-dimensional numerical linear algebra problem. Final PageRank vector is stationary distribution of this Markov chain. In numerical terms, we want vector that does not change after applying PageRank update.

3.2 Power Iteration

Main algorithm used in this project is power iteration. Starting from initial probability vector,

$$r^{(0)} = \frac{1}{n} \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix},$$

we repeatedly compute

$$r^{(k+1)} = \alpha S r^{(k)} + \alpha m_D^{(k)} v + (1 - \alpha) v.$$

The uniform initial vector is standard choice because it is already valid probability distribution. From point of view of power method, initial vector must have nonzero component in direction of

dominant eigenvector. Since the dominant PageRank vector is positive, the uniform starting vector is natural choice.

The choice of starting vector can affect number of iterations, mostly in early steps. I use uniform vector because it is positive, normalized, and does not favor any page before computation. A different positive probability vector would still converge to same PageRank vector, but it may require slightly different number of iterations. A very uneven initial vector can make early behavior less smooth, but teleportation reduces this sensitivity.

Iteration is stopped when change between two successive approximations is small:

$$\|r^{(k+1)} - r^{(k)}\|_1 < \text{TOL}.$$

In MATLAB implementation, I use

$$\text{TOL} = 10^{-10}.$$

The 1-norm is natural here because PageRank is probability vector, and sum of all entries should stay equal to one.

3.3 Algorithm Steps

The computational algorithm is:

1. Build sparse adjacency matrix A from index vectors.
2. Compute number of outgoing links for each page.
3. Construct sparse transition matrix using $S = AD^{-1}$.
4. Identify dangling nodes and keep their columns zero in S.
5. Choose damping factor α and teleportation vector v.
6. Start with uniform rank vector $r^{(0)}$.
7. Apply sparse power iteration with implicit dangling-node correction.
8. Stop when $\|r^{(k+1)} - r^{(k)}\|_1 < 10^{-10}$.
9. Rank pages from largest PageRank value to smallest.

This algorithm is simple, but it is powerful because it only needs sparse matrix-vector multiplication. For large sparse web graph, this is much more practical than forming dense matrix and inverting it.

4 Connection to Numerical Analysis Topics

4.1 Eigenvalue Problems

PageRank can be interpreted as eigenvalue problem. For a matrix G, an eigenvector is a nonzero vector whose direction does not change after multiplication by G. In formula form,

$$Gx = \lambda x.$$

Here x is eigenvector and λ is eigenvalue. The matrix may stretch or shrink this vector, but it does not move it to completely different direction.

In PageRank, important vector is dominant eigenvector. Dominant eigenvector is the eigenvector corresponding to largest eigenvalue in magnitude. For stochastic matrix used in PageRank, dominant eigenvalue is

$$\lambda_1 = 1.$$

The PageRank vector is this dominant eigenvector because it satisfies

$$Gr = r.$$

This means that after applying Google matrix, PageRank vector stays the same. In random surfer interpretation, this is stationary distribution: after long time, probability of being on each page no longer changes.

If full Google matrix is written as

$$G = \alpha S_{\text{full}} + (1 - \alpha) v e^T,$$

where e is vector of all ones, then PageRank vector satisfies $r = Gr$. This connects directly to eigenvalue problems in Numerical Analysis. Instead of finding all eigenvalues and eigenvectors, we only need dominant eigenvector. Power method is designed for this type of problem.

Since the Google matrix G is column-stochastic, each column sums to one. Therefore 1 is an eigenvalue of G, and its spectral radius satisfies

$$\rho(G) = 1.$$

The Perron-Frobenius theorem explains why this dominant vector is meaningful here. In simple

terms, for a positive stochastic matrix, there is one main positive eigenvector associated with dominant eigenvalue. In PageRank, teleportation makes Google matrix positive, because every page has some probability of being reached. Because of this, PageRank vector is unique and has positive entries.

The damping factor also controls the remaining eigenvalues. Informally, the non-dominant part of link-following behavior is scaled by α, so the magnitude of subdominant eigenvalues is bounded by damping factor in standard PageRank model. This is why larger α usually gives slower convergence: second eigenvalue can be closer to 1, and spectral gap becomes smaller.

This matters numerically because power iteration needs stable dominant direction to converge to. The theorem gives mathematical reason why repeated application of Google matrix leads to one well-defined ranking vector.

4.2 Iterative Methods

Power method is iterative algorithm. This connects to part of course where iterative solvers and eigenvalue problems were studied. Instead of solving system in one direct step, method creates sequence of approximations:

$$r^{(0)}, r^{(1)}, r^{(2)}, \ldots$$

and stops once approximation is accurate enough.

This is similar in spirit to stationary iterative methods such as Jacobi and Gauss-Seidel because each new approximation is obtained by applying matrix-based update to previous approximation. Difference is that PageRank is usually interpreted as eigenvector or fixed-point problem, not only standard linear system solve. Still, same numerical ideas appear: initial guess, repeated iteration, convergence behavior, and stopping tolerance.

4.3 Sparse Matrices

Hyperlink matrix is sparse because each web page links only to small number of other pages compared with total size of web. This is important computationally. If n is very large, storing dense $n \times n$ matrix is impossible. For example, dense matrix with millions of rows and columns would require too much memory.

Sparse matrix storage avoids storing zero entries. Computational cost of multiplying S by r depends mainly on number of nonzero links, not on n^2. This is one reason why iterative methods are necessary in search engine systems.

In this experiment, adjacency matrix A has 23 nonzero entries out of $10 \times 10 = 100$ possible entries.

This small example already shows sparse structure of hyperlink graph. In real web graph, difference between actual links and all possible links becomes much larger.

In MATLAB code, sparse transition matrix is not constructed by repeatedly modifying individual sparse columns. Instead, non-dangling columns are normalized by multiplying adjacency matrix by sparse diagonal matrix:

$$S = AD^{-1}.$$

This is implemented using `spdiags`. Dangling nodes are not handled by filling columns with dense uniform vectors. Instead, rank mass from dangling nodes is redistributed during PageRank iteration. This distinction is important because explicitly filling dangling-node columns would destroy sparsity for large graphs.

5 Numerical Experiment and Results

5.1 Example Web Graph

For numerical experiment, I use small directed graph with 10 pages and 23 directed links. The graph is not meant to represent real web. It is only test case that makes PageRank calculation visible and easy to check.

The graph includes one dangling node, page 10, which has no outgoing links. This is useful because implementation can test dangling-node correction. Graph is small, but still nontrivial: several pages have multiple incoming links, and ranking is not determined only by simple in-degree.

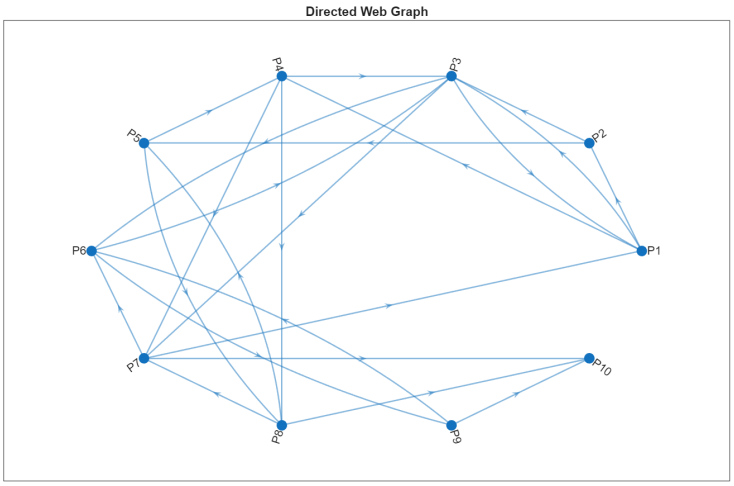

Figure 1: Directed web graph used for PageRank experiment. The graph is intentionally small, but it includes multiple incoming links and one dangling node.

For main experiment, I use

$$n = 10, \qquad \alpha = 0.85, \qquad \text{TOL} = 10^{-10}.$$

The transition matrix has 23 nonzero entries. The maximum column-sum error for non-dangling columns was

$$0.000 \times 10^{0},$$

which confirms that sparse transition matrix was constructed correctly.

5.2 Computed PageRank Values

Sparse power iteration converged in 27 iterations. Final L_1 change was

$$9.474 \times 10^{-11},$$

which is below stopping tolerance. Sum of final PageRank vector was

$$1.000000000000.$$

The computed PageRank values are shown in Table 1 and Figure 2.

Table 1: Computed PageRank values sorted from highest to lowest.

Page	In-Degree	Out-Degree	Dangling	PageRank
3	4	3	No	0.15920
6	3	2	No	0.13819
7	3	3	No	0.11572
10	3	0	Yes	0.11522
1	2	3	No	0.10269
4	2	3	No	0.08357
9	1	2	No	0.08352
8	2	3	No	0.07816
5	2	2	No	0.06984
2	1	2	No	0.05389

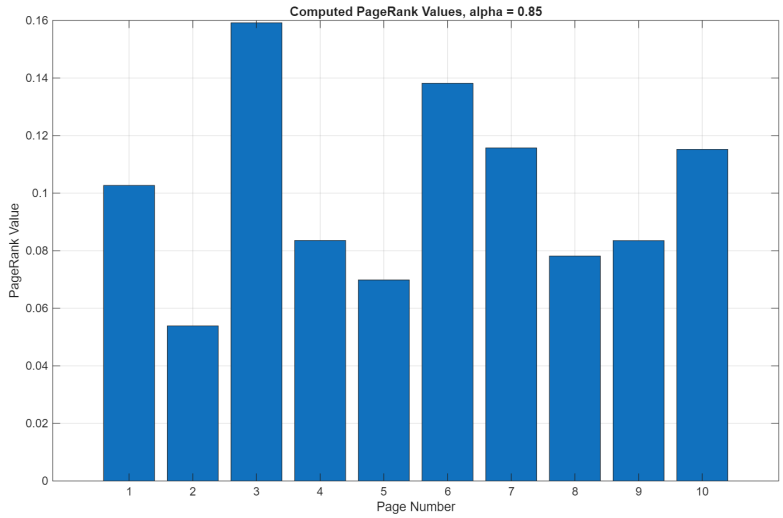

Figure 2: Computed PageRank values for the 10-page example.

Page 3 receives highest PageRank value, about 0.15920. This is reasonable because it has largest in-degree and receives links from several other pages. Page 10 is dangling node, but it still has relatively high PageRank value, about 0.11522, because pages 7, 8, and 9 point to it. This shows why PageRank is not same as simply counting incoming or outgoing links. Location of page inside link structure matters.

5.3 Convergence of Power Iteration

Convergence history is shown in Figure 3. Error decreases from initial value to below 10^{-10} in 27 iterations.

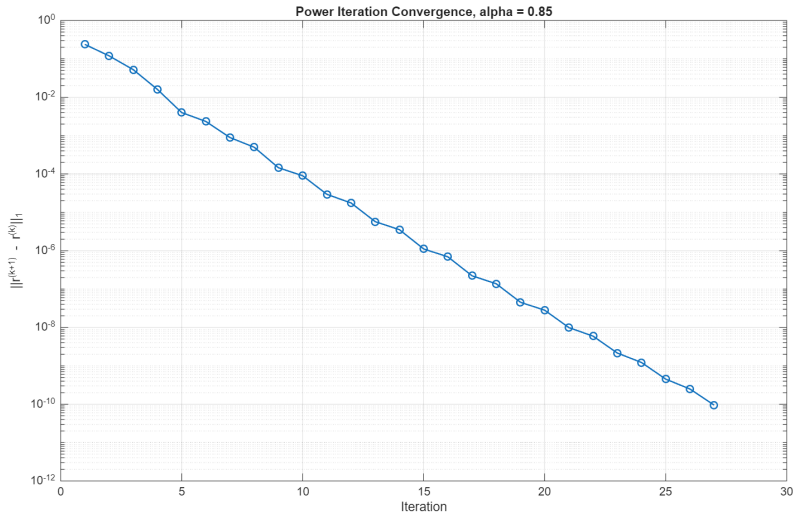

Figure 3: Convergence history of power iteration method for $\alpha = 0.85$.

The semilog plot is close to straight decreasing line after first few steps. This is typical for power iteration because error often decreases geometrically when there is clear spectral gap.

To make convergence result more quantitative, I also fitted a line to the $\log_{10}$ convergence error after first few iterations. The estimated slope was approximately

$$a \approx -0.3482.$$

This corresponds to error reduction factor

$$10^{-0.3482} \approx 0.4486$$

per iteration. In other words, after early transient steps, each new iteration reduced error to about 44.86% of previous error. This supports the visual observation that convergence is close to geometric on the semilog plot.

5.4 Effect of Damping Factor

Damping factor α has both modeling and numerical meaning. From modeling side, α is probability that random surfer follows hyperlink, while $1 - \alpha$ is probability of teleporting to another page.

From numerical side, α affects convergence rate of power iteration. For Google matrix, dominant eigenvalue is

$$\lambda_1 = 1.$$

The remaining eigenvalues are controlled by damping factor. For power iteration, convergence depends on how strongly subdominant eigenvalues remain after repeated multiplication. In practical terms, larger α usually makes subdominant eigenvalues larger in magnitude, which reduces spectral gap and slows convergence.

This behavior appears in numerical experiment. As α increases from 0.70 to 0.95, number of iterations increases from 22 to 32.

Table 2: Effect of damping factor on convergence.

Damping Factor α	Iterations	Final Error
0.70	22	8.2887×10^{-11}
0.85	27	9.4740×10^{-11}
0.95	32	8.0424×10^{-11}

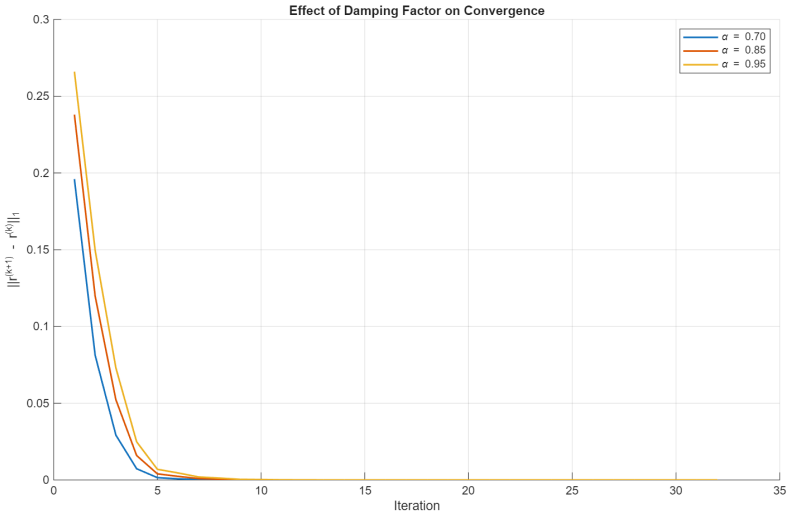

Figure 4: Comparison of convergence for different damping factors.

Final error values in Table 2 are all close to 10^{-10}, so comparison is fair. Increase in iteration count shows that α is not only modeling parameter, but also numerical parameter.

5.5 Verification by Direct Solve

As verification step, I also solved equivalent linear system directly for small $n = 10$ example:

$$(I - \alpha S_{\text{full}})r = (1 - \alpha)v.$$

This direct solve is not intended for large web-scale graphs, but it is useful for confirming that iterative method solves correct fixed-point problem.

Difference between sparse power iteration result and direct solve result was

$$\|r_{\text{power}} - r_{\text{direct}}\|_1 = 5.539 \times 10^{-11}.$$

The direct solve probability mass error was

$$\left| \sum_i r_{\text{direct},i} - 1 \right| = 0.$$

These values confirm that power iteration result agrees with direct linear solve up to selected tolerance.

6 Potential Numerical Challenges

6.1 Slow Convergence

One numerical challenge is slow convergence. Power method converges quickly when dominant eigenvalue is well separated from second-largest eigenvalue in magnitude. However, when second eigenvalue is close to dominant one, convergence becomes slower.

For PageRank, damping factor affects this behavior. Larger values of α make ranking depend more strongly on link graph, but they also tend to slow convergence. This is tradeoff between ranking sensitivity and computational cost.

6.2 Dangling Nodes

Dangling nodes are another important numerical issue. Without correction, page with no outgoing links creates zero column in transition matrix. This breaks probability interpretation of Markov chain.

Numerical fix used in this project is not to fill dangling column explicitly. Instead, total rank mass on dangling nodes is computed at each iteration and redistributed uniformly. This keeps sparse matrix structure intact while preserving total probability.

6.3 Large Sparse Matrix Storage

For real web-scale systems, matrix size can be enormous. Direct dense matrix representation is not realistic. Numerical Analysis helps here through sparse matrix methods. Instead of storing all entries of S, we only store nonzero hyperlink entries.

This changes practical feasibility of algorithm. Dense method would scale like $O(n^2)$ in storage, but sparse storage scales closer to number of actual links. Since web graph is sparse, this is essential.

The use of $S = AD^{-1}$ also matters computationally. It builds transition matrix using vectorized sparse matrix operation rather than slow column-by-column sparse modification. This is not just coding style choice. It reflects the fact that numerical algorithms must be designed around memory access and sparse matrix structure.

6.4 Stopping Tolerance

Stopping tolerance controls balance between accuracy and runtime. A loose tolerance may stop algorithm too early and give inaccurate rankings. A very strict tolerance may require many extra iterations without changing practical order of top pages.

In this report, I use

$$\|r^{(k+1)} - r^{(k)}\|_1 < 10^{-10}.$$

This is stricter than needed for many practical applications, but it makes numerical behavior clear.

6.5 Probability Mass and Round-Off Drift

In exact arithmetic, PageRank update preserves total probability mass. Since $r^{(k)}$ is probability vector, entries should satisfy

$$\sum_{i=1}^{n} r_i^{(k)} = 1$$

at every iteration. If transition matrix and dangling-node correction are implemented correctly, next iterate should also sum to one.

However, in floating-point arithmetic, very small round-off drift may appear after repeated matrix-vector operations. MATLAB code records quantity

$$\left| \sum_{i=1}^{n} r_i^{(k+1)} - 1 \right|$$

before renormalization. This check is included to make sure that renormalization is not hiding algorithmic mistake. In experiment, maximum mass error before renormalization was

$$2.220 \times 10^{-16}.$$

This is at level of machine precision. The mass-error figure below is plotted directly from `massErrors`, without adding `eps`, so the displayed scale matches the numerical maximum.

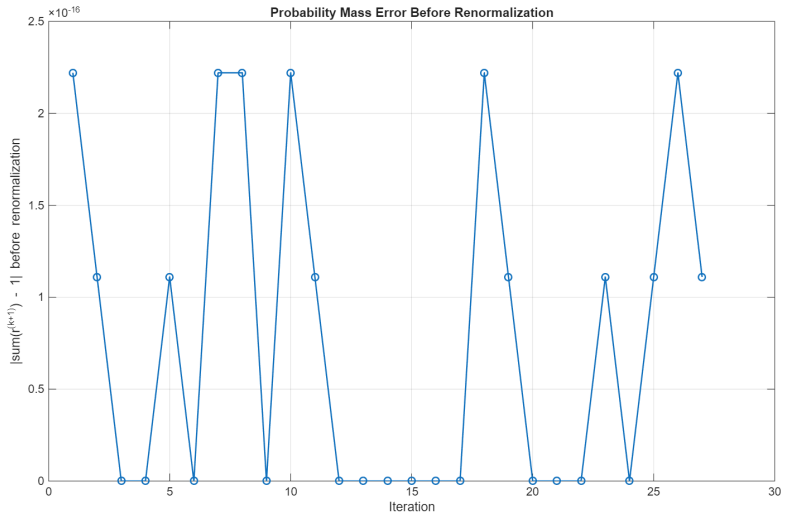

Figure 5: Probability mass error before renormalization during PageRank iteration.

The probability mass error remains extremely small throughout iteration. This confirms that PageRank update preserves stochastic structure of problem and renormalization only corrects floating-point round-off drift.

6.6 Limitation of Runtime Scaling

A limitation of this project is that numerical experiment uses small graph with $n = 10$. Because of this, runtime scaling is not fully demonstrated. For larger extension, one should generate sparse graphs of increasing size and compare sparse power iteration with direct solve in runtime and memory.

I did not include full runtime scaling experiment because main goal here was to show formulation, algorithm, and numerical correctness on inspectable example. Still, the structure of implementation already follows scalable approach: sparse matrix storage, sparse matrix-vector multiplication, diagonal scaling, and implicit dangling-node correction.

7 Conclusion

In this project, PageRank was studied as application of Numerical Analysis to computer systems. The problem begins as search engine ranking problem, but mathematically it becomes numerical linear algebra problem. By modeling web as directed graph, constructing stochastic matrix, and applying power iteration method, we can approximate PageRank vector.

This project connects strongly with several topics from Numerical Analysis: eigenvalue problems, iterative methods, convergence, sparse matrices, stopping criteria, and numerical stability. PageRank vector is dominant eigenvector of Google matrix, and power method provides practical way to compute it without direct matrix inversion.

Numerical experiment showed that sparse power iteration converged in 27 iterations for $\alpha = 0.85$, with final L_1 change of 9.474×10^{-11}. The fitted slope of the $\log_{10}$ convergence curve was approximately -0.3482, which corresponds to error reduction factor about 0.4486 per iteration after early steps. Final PageRank vector summed to one, and agreement with direct linear solve was within 5.539×10^{-11}. Damping-factor experiment also showed expected numerical behavior: larger values of α required more iterations.

Final MATLAB implementation also follows computational structure of problem. The adjacency matrix is assembled from index vectors, transition matrix is constructed using sparse diagonal scaling, dangling nodes are handled implicitly, and probability mass conservation is checked before renormalization. These details are important because PageRank is not only mathematical eigenvector problem, but also large-scale numerical computation problem.

Overall, PageRank is strong example of applied mathematics inside real computational system. A search engine problem becomes matrix problem, and solution depends on numerical algorithms that are efficient, stable, and scalable.

References

[1] L. Page, S. Brin, R. Motwani, and T. Winograd, *The PageRank Citation Ranking: Bringing Order to the Web*, Stanford InfoLab Technical Report, 1999.

[2] A. N. Langville and C. D. Meyer, *Google's PageRank and Beyond: The Science of Search Engine Rankings*, Princeton University Press, 2006.

[3] R. L. Burden and J. D. Faires, *Numerical Analysis*, Brooks/Cole, 9th edition, 2011.

[4] C. Moler, *The World's Largest Matrix Computation*, MathWorks Newsletter, 2002.

A MATLAB Code

The complete MATLAB code used for this report is included below. It constructs graph, computes PageRank by sparse power iteration, compares damping factors, estimates convergence slope, checks probability mass conservation, and saves figures used in report.

```
%% Application of Numerical Linear Algebra to PageRank
% MTH 4311 - Numerical Analysis
% Artem Okhten

clear;
clc;
close all;

%% Setup

n = 10;
alpha = 0.85;
TOL = 1e-10;
maxIter = 1000;

v = ones(n, 1) / n;

fprintf("n = %d, alpha = %.2f, tolerance = %.1e\n\n", n, alpha, TOL);

%% Sparse Web Graph

% A(i,j)=1 means page j links to page i.
% Page 10 is left as a dangling node.

row_idx = [ ...
    2, 3, 4, ...
    3, 5, ...
    1, 6, 7, ...
    3, 7, 8, ...
    4, 8, ...
    3, 9, ...
    1, 6, 10, ...
    5, 7, 10, ...
    6, 10 ...
];

col_idx = [ ...
    1, 1, 1, ...
    2, 2, ...
    3, 3, 3, ...
    4, 4, 4, ...
    5, 5, ...
    6, 6, ...
    7, 7, 7, ...
    8, 8, 8, ...
    9, 9 ...
];

A = sparse(row_idx, col_idx, 1, n, n);
```

```
50
51 fprintf("Nonzero links in A: %d\n", nnz(A));
52
53 %% Transition Matrix
54
55 [S, dangling, outDegree] = build_sparse_transition_matrix(A);
56
57 columnSums = full(sum(S, 1));
58 nonDanglingColumns = ~dangling;
59 maxColumnError = max(abs(columnSums(nonDanglingColumns) - 1));
60
61 fprintf("Nonzero entries in S: %d\n", nnz(S));
62 fprintf("Dangling pages: ");
63 disp(find(dangling));
64 fprintf("Max column-sum error: %.3e\n\n", maxColumnError);
65
66 %% Graph Figure
67
68 G = digraph(A');
69
70 figure;
71 p = plot(G, ...
72     'Layout', 'circle', ...
73     'NodeLabel', compose('P%d', 1:n));
74
75 title('Directed Web Graph');
76 p.MarkerSize = 8;
77 p.LineWidth = 1.2;
78
79 saveas(gcf, 'pagerank_graph.png');
80
81 %% Power Iteration
82
83 [r_power, errors, massErrors, numIter] = pagerank_power_sparse( ...
84     S, dangling, alpha, v, TOL, maxIter);
85
86 fprintf("Power iteration iterations: %d\n", numIter);
87 fprintf("Final L1 change: %.3e\n", errors(end));
88 fprintf("Sum of PageRank vector: %.12f\n", sum(r_power));
89 fprintf("Max mass error before renormalization: %.3e\n\n", max(massErrors));
90
91 %% Results Table
92
93 pageNumbers = (1:n)';
94 inDegree = full(sum(A, 2));
95 outDegreeColumn = outDegree(:);
96
97 resultsTable = table( ...
98     pageNumbers, ...
99     inDegree, ...
100    outDegreeColumn, ...
101    dangling(:), ...
102    r_power, ...
103    'VariableNames', {'Page', 'InDegree', 'OutDegree', 'IsDangling', 'PageRank'} ...
104 );
105
```

```
rankedTable = sortrows(resultsTable, 'PageRank', 'descend');

disp("PageRank results:");
disp(rankedTable);

%% PageRank Values Figure

figure;
bar(1:n, r_power);
xlabel('Page Number');
ylabel('PageRank Value');
title(sprintf('Computed PageRank Values, alpha = %.2f', alpha));
grid on;

saveas(gcf, 'pagerank_values.png');

%% Convergence Figure

figure;
semilogy(1:length(errors), errors, '-o', 'LineWidth', 1.2);
xlabel('Iteration');
ylabel('||r^{(k+1)} - r^{(k)}||_1');
title(sprintf('Power Iteration Convergence, alpha = %.2f', alpha));
grid on;

saveas(gcf, 'pagerank_convergence.png');

%% Convergence Slope Estimate

iterIndex = (1:length(errors))';

% Skip first few points because early iterations are less linear.
fitStart = 5;

p = polyfit(iterIndex(fitStart:end), log10(errors(fitStart:end)), 1);

convSlope = p(1);
convFactor = 10^convSlope;

fprintf("Convergence slope on log10 scale: %.4f\n", convSlope);
fprintf("Approximate error factor per iteration: %.4f\n\n", convFactor);

%% Damping Factor Comparison

alphaValues = [0.70, 0.85, 0.95];

allErrors = cell(length(alphaValues), 1);
allIterations = zeros(length(alphaValues), 1);
finalErrors = zeros(length(alphaValues), 1);

for i = 1:length(alphaValues)
    currentAlpha = alphaValues(i);

    [~, errHist, ~, iters] = pagerank_power_sparse( ...
        S, dangling, currentAlpha, v, TOL, maxIter);

```

```
    allErrors{i} = errHist;
    allIterations(i) = iters;
    finalErrors(i) = errHist(end);
end

alphaTable = table(alphaValues(:), allIterations(:), finalErrors(:), ...
    'VariableNames', {'Alpha', 'Iterations', 'FinalError'});

disp("Damping factor comparison:");
disp(alphaTable);

figure;
hold on;

for i = 1:length(alphaValues)
    semilogy(1:length(allErrors{i}), allErrors{i}, ...
        'LineWidth', 1.4, ...
        'DisplayName', sprintf('\\alpha = %.2f', alphaValues(i)));
end

xlabel('Iteration');
ylabel('||r^{(k+1)} - r^{(k)}||_1');
title('Effect of Damping Factor on Convergence');
legend('Location', 'northeast');
grid on;
hold off;

saveas(gcf, 'pagerank_alpha_comparison.png');

%% Direct Solve Check

S_full = full(S);

for j = 1:n
    if dangling(j)
        S_full(:, j) = v;
    end
end

I = eye(n);
r_direct = (I - alpha*S_full) \ ((1 - alpha) * v);

directMassError = abs(sum(r_direct) - 1);
directDifference = norm(r_direct - r_power, 1);

fprintf("Direct solve L1 difference: %.3e\n", directDifference);
fprintf("Direct solve mass error: %.3e\n\n", directMassError);

comparisonTable = table( ...
    pageNumbers, ...
    r_power, ...
    r_direct, ...
    abs(r_power - r_direct), ...
    'VariableNames', {'Page', 'PowerIteration', 'DirectSolve', 'AbsoluteDifference'} ...
);

```

```
disp("Direct solve comparison:");
disp(comparisonTable);

%% Probability Mass Figure

figure;
plot(1:length(massErrors), massErrors, '-o', 'LineWidth', 1.2);
xlabel('Iteration');
ylabel('|sum(r^{(k+1)}) - 1| before renormalization');
title('Probability Mass Error Before Renormalization');
grid on;

saveas(gcf, 'pagerank_mass_error.png');

%% Local Functions

function [S, dangling, outDegree] = build_sparse_transition_matrix(A)

    n = size(A, 1);

    outDegree = full(sum(A, 1));
    dangling = (outDegree == 0);

    invOutDegree = zeros(1, n);
    invOutDegree(~dangling) = 1 ./ outDegree(~dangling);

    Dinv = spdiags(invOutDegree(:), 0, n, n);
    S = A * Dinv;

end

function [r, errors, massErrors, iter] = pagerank_power_sparse(S, dangling, alpha, v, TOL,
    maxIter)

    r = v;

    errors = zeros(maxIter, 1);
    massErrors = zeros(maxIter, 1);

    for k = 1:maxIter
        danglingMass = sum(r(dangling));

        r_new = alpha * (S * r) ...
              + alpha * danglingMass * v ...
              + (1 - alpha) * v;

        massErrors(k) = abs(sum(r_new) - 1);

        r_new = r_new / sum(r_new);

        err = norm(r_new - r, 1);
        errors(k) = err;

        r = r_new;

        if err < TOL
```

```
            iter = k;
            errors = errors(1:k);
            massErrors = massErrors(1:k);
            return;
        end
    end

    iter = maxIter;
    errors = errors(1:maxIter);
    massErrors = massErrors(1:maxIter);

end
```

Listing 1: MATLAB code for PageRank numerical experiment.